Summaries

Polymorphism.-

The definition of polymorphism is of the greek for "having many form". It's one of the three pillars of OOP, along with encapsulation and inheritance. The idea is that client code treats related object as the same general type, and at runtime each object runs it's own correct version of the behavior

Without polymorphism, client code checks each object's type with instanceof, casts it, and calls a different method for each piece (getPawnMoves(), getRookMoves()..)
The code works, but it is:

- Hard to maintain, because every new piece means another if/else.
- Error-prone and verbose.
- Built around asking objects what they are, instead of telling them what to do.

Solution

* Establish a common interface: declare abstract Location[] getPossibleMoves[] in the superclass ChessPiece.
* Implement it in each subclass (Pawn, Rook) with @Override.
* Depend on the superclass in the clean code.

In Java

With objects: a superclass references can hold a subclass object. ArrayList<FarmAnimal> accepts a Chicken, but ArrayList<Chicken> doesn't accept a FarmAnimal

With methods:

- Overriding: a subclass redefines an inherited method with same signature, e.g. Chicken overrides computeFoodCost(). The version that runs is chosen at runtime
- Overloading: several methods in the same class share a name but take different parameters. The version is chosen at compile time.

Abstraction.-

Keeping only the parts of an object that matter for a given purpose and ignoring the rest.

- Abstraction (verb): the process of creating an interface for a component that exposes what users need and hides everything else.
- An abstraction (noun): the resulting definition with unnecesary details left out. A class definition is an abstraction

  - Public members are the details that are exposed
  - Private members and method implementations are the hidden details.
  - Irrelevant details are left out of the class entirely.

Desing with abstraction: Find what different systems have in common (moving on a 2D grid, picking up items, user-controlled objects) and put it in an abstract layer. Leave the specific differences to the concrete subclasses. The abstract layer can't stand

on its own, wich is why it's abstract.

Interfaces In Java

- A contract: It lists what a class must do, not how it does it.
- A class uses implements to adopt an interface, and it can implement several.
- An interface can extends other interfaces.
- It cannot be instantiated, but it can be used as a references type

	Abstract class	Interface
Instantiable	No	No
Keyword	extends	implements
How many per class	1	Several
Attributes / state	Yes	Constants only
Best for	An "is-a" relationship	A "can-do" capability